FICHE APPRENANT · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 4

Inventaire réseau (CRUD)

Durée : 5 jours CRUD Validation Formulaires Requêtes efficaces (N+1)

Objectifs de la semaine

- maîtriser le **CRUD** Laravel (Create, Read, Update, Delete) ;
- **valider** les données reçues et afficher les erreurs proprement ;
- réutiliser un seul formulaire pour créer et modifier ;
- repérer et corriger le problème des **requêtes N+1**.

1. Les contrôleurs « resource »

```
php artisan make:controller SiteController --resource
```

```
Route::resource('sites', SiteController::class)->except('show');
```

Une seule ligne crée toutes les routes :

Verbe	URL	Méthode	Rôle
GET	/sites	index	liste
GET	/sites/create	create	formulaire vide
POST	/sites	store	enregistrer
GET	/sites/{site}/edit	edit	formulaire rempli
PUT	/sites/{site}	update	mettre à jour
DELETE	/sites/{site}	destroy	supprimer

Route model binding : en écrivant `public function edit(Site $site)`, Laravel cherche tout seul le site dont l'id est dans l'URL, et renvoie une erreur 404 s'il n'existe pas.

2. Valider les données

Règle n°1 du développement web : ne jamais faire confiance à ce qui vient du navigateur.

```
$donnees = $request->validate([
    'code' => ['required', 'string', 'max:20', Rule::unique('sites')->ignore($site)],
    'nom' => ['required', 'string', 'max:255'],
    'type' => ['required', Rule::in(Site::TYPES)],
]);

Site::create($donnees);
```

- Si la validation échoue, Laravel **revient automatiquement** au formulaire avec les erreurs et les anciennes valeurs.
- `Rule::unique('sites')->ignore($site)` : le code doit être unique, **sauf** pour le site en cours de modification.

Règles utiles cette semaine : `required`, `nullable`, `max`, `exists:sites,id`, `ip`, `different:site_a_id`, `Rule::in([...])`.

3. Un formulaire pour créer ET modifier

```
<form method="POST" action="{{ $site->exists ? route('sites.update', $site) :
route('sites.store') }}">
    @csrf
    @if ($site->exists) @method('PUT') @endif

    <input type="text" name="code" value="{{ old('code', $site->code) }}"
           class="form-control @error('code') is-invalid @enderror">
    @error('code')<div class="invalid-feedback">{{ $message }}</div>@enderror
</form>
```

- `$site->exists` est `false` pour un `new Site`, `true` pour un site venant de la base.
- `old('code', $site->code)` : après une erreur, on réaffiche ce que l'utilisateur avait tapé.
- Les formulaires HTML ne connaissent que GET et POST : `@method('PUT')` simule les autres verbes.

4. Le piège du N+1

```
@foreach ($equipements as $equipement)
    {{ $equipement->site->code }}
@endforeach
```

Avec 50 équipements, ce code exécute **51 requêtes** : 1 pour la liste, puis 1 par équipement pour aller chercher son site. C'est le problème **N+1**.

Solution, le **chargement anticipé** :

```
$equipements = Equipement::with('site')->get();   // 2 requêtes au total
```

Pour compter sans charger : `Site::withCount('equipements')->get()` ajoute un attribut `equipements_count`.

> **Astuce** — Pour voir les requêtes exécutées, ajoute temporairement dans `AppServiceProvider::boot()` : `DB::listen(fn ($q) => logger($q->sql));` puis regarde `storage/logs/laravel.log`.

Exercices MySQL

1. Écrire la requête SQL qu'exécute `Site::withCount('equipements')` (indice : sous-requête ou `LEFT JOIN … GROUP BY`).
2. Quelle différence entre `JOIN` et `LEFT JOIN` ? Ajoute un site sans équipement et compare les résultats.
3. Lister les liaisons qui partent **ou** arrivent à `BKO-DC1`.
4. Mettre à jour en SQL le statut d'un équipement à `maintenance`, puis vérifier dans l'application.

Travail demandé

1. **Sites** : liste (code, nom, ville, type, nombre d'équipements, nombre d'incidents actifs), création, modification, suppression.
   - Refuser la suppression d'un site qui a des équipements ou des incidents, avec un message clair.
2. **Équipements** : liste paginée, recherche par hostname, filtre par site ; type et constructeur en listes déroulantes ; IP validée.
3. **Liaisons** : liste « BKO-DC1 ⇄ KTI-POP1 » ; site A et site B différents ; statut (active, dégradée, coupée) en badge coloré.
4. Créer un composant `<x-bouton-supprimer>` avec confirmation JavaScript.
5. Traduire les messages de validation dans `lang/fr/validation.php`.
6. Faire un commit par écran terminé.

Livrable

Les trois écrans d'inventaire fonctionnent de bout en bout.

Auto-évaluation

- [ ] Un code de site en double est refusé à la création, accepté en modifiant le même site
- [ ] Une IP invalide (`10.10.1.300`) est refusée
- [ ] Une liaison d'un site vers lui-même est refusée
- [ ] Mes listes n'exécutent pas une requête par ligne
- [ ] La suppression demande confirmation et utilise `DELETE`
- [ ] Les messages d'erreur sont en français
- [ ] Je sais expliquer le problème N+1 avec un exemple

Ressources

- Laravel — Contrôleurs resource : laravel.com/docs/controllers#resource-controllers
- Laravel — Validation : laravel.com/docs/validation
- Laravel — Eager loading : laravel.com/docs/eloquent-relationships#eager-loading

- MySQL — LEFT JOIN : dev.mysql.com/doc/refman/8.4/en/join.html

Pour aller plus loin

Créer une page de détail d'un site avec ses équipements, ses liaisons et ses 10 derniers incidents.